

MANAGING TASK ERRORS IN PLAYS:

- Ansible evaluates the return code of each task to determine whether the task succeeded or failed.
- Normally, when a task fails Ansible immediately aborts the rest of the play on that host, skipping all subsequent tasks.
- But sometimes you might want to have play execution continue even if a task fails.
- For example, you might expect that a particular task could fail, and you might want to recover by running some other tasks conditionally.
- There are a number of Ansible features that can be used to manage task errors.

1-Ignoring Task Failure:

- By default, if a task fails, the play is aborted. However, this behavior can be overridden by ignoring failed tasks.
- You can use the `ignore_errors` keyword in a task to accomplish this.

EXAMPLE:

- name: Install Latest Packages

hosts: rhelnode

gather_facts: no

tasks:

- name: install xyz package

yum:

name: xyz

state: latest

ignore_errors: yes

- name: install new package

yum:

name: curl

state: latest

- ✓ first task will fail as "xyz" is not a package
- ✓ So, play will be aborted if `"ignore_errors: yes"` is not mentioned or commented.

2-Forcing Execution of Handlers after Task Failure

- Normally when a task fails and the play aborts on that host, any handlers that had been notified by earlier tasks in the play will not run.
- If you set the `force_handlers: yes` keyword on the play, then notified handlers are called even if the play aborted because a later task failed.

- name: Test Force Handlers

gather_facts: no

hosts: rhelnode

force_handlers: yes

tasks:

- name: Install nginx

yum:

name: nginx

state: present

notify: restart nginx

- name: Simulate a failure

command: /bin/false

handlers:

- name: restart nginx

service:

name: nginx

state: restarted

Best Practice

- Use `force_handlers: yes` sparingly. It's powerful but can mask failures if you rely on handlers to "fix" things.
- Combine with `ignore_errors` or `failed_when` for fine-grained control.

3-Specifying Task Failure Conditions:

- You can use the **failed_when** keyword on a task to specify which conditions indicate that the task has failed.
- This is often used with command modules that may successfully execute a command, but the command's output indicates a failure.
- **failed_when** is one of the most powerful Ansible directives because it lets you define custom failure conditions for a task.
- Instead of relying only on the command's exit code, you can decide when a task should be considered failed (or not).

```yaml

- name: Demo of failed\_when usage

hosts: rhelnode

gather\_facts: no

tasks:

- name: Run a health check script

command: /usr/local/bin/healthcheck.sh

register: health\_result

changed\_when: false

failed\_when:

- "'ERROR' in health\_result.stdout"

- health\_result.rc != 0

- name: Show health check output

debug:

var: health\_result.stdout

#### Best practice:

- Use **failed\_when** for fine-grained control instead of blindly using **ignore\_errors**.
- Always combine with **register** to capture task results.
- Keep conditions explicit and readable for maintainability.

#### 4: Specifying When a Task Reports “Changed” Results

- When a task makes a change to a managed host, it reports the changed state and notifies handlers.
- When a task does not need to make a change, it reports ok and does not notify handlers.
- The `changed_when` keyword can be used to control when a task reports that it has changed.
- In Ansible, the `changed_when` directive lets you control whether a task is marked as changed or not.
- By default, Ansible decides based on the module’s behavior (e.g., copy marks changed if the file differs).
- With `changed_when`, you override that logic, its useful for idempotency and cleaner playbook run.
- `changed_when: false` keeps validation tasks from cluttering your playbook output with “`changed`” states.
- Conditional `changed_when` lets you highlight important state changes (like high disk usage) without actually modifying anything.
- Combining with `failed_when` gives you precise control over both change detection and failure logic.

#### PlayBook-1

```
``yaml
```

```
- name: Demonstrate changed_when usage
```

```
hosts: rhelnode
```

```
become: yes
```

```
tasks:
```

```
- name: Ensure nginx is installed
```

```
 yum:
```

```
 name: nginx
```

```
 state: present
```

```
- name: Validate nginx configuration
```

```
 command: nginx -t
```

```
 register: nginx_test
```

```
 changed_when: false # Validation should never mark as changed
```

```
 failed_when: "'successful' not in nginx_test.stderr"
```

```
- name: Check disk usage
```

```
 shell: df -h / | awk 'NR==2 {print $5}' | sed 's/%/'
```

```
 register: disk_usage
```

```
changed_when: disk_usage.stdout|int > 80 # Mark changed if usage > 80%
```

```
failed_when: disk_usage.stdout|int > 90 # Fail if usage > 90%
```

```
- name: Debug disk usage
```

```
debug:
```

```
msg: "Disk usage is {{ disk_usage.stdout }}%"
```

```
'''
```

#### **EXAMPLE-2 ; changed\_when: false:**

```
---yaml
```

```
- name: Validate NGINX service
```

```
hosts: rhelnode
```

```
gather_facts: no
```

```
tasks:
```

```
- name: Check NGINX status
```

```
command: systemctl status nginx
```

```
register: nginx_status
```

```
changed_when: false
```

```
failed_when: "'inactive' in nginx_status.stdout"
```

```
- name: Show NGINX status output
```

```
debug:
```

```
var: nginx_status.stdout
```

```
'''
```

